Functional Specification for Parallel go-ethereum(geth) project

Author	Itay Elam, Ido Talker
Version	V1.0
Created	06/04/2026
Last Updated	15/04/2026

1 Overview

Geth is one of the execution clients in the Ethereum ecosystem, written in Go programming language. Ethereum's execution model, for different reasons, relies heavily on sequential execution of transactions in a block. This behaviour is one of the main scalability bottlenecks in the Ethereum protocol and our aim is to find, examine and compare different parallel execution models for a single block execution process.

This project comes as a direct continuation of Itay's preliminary work.

1.1 Detailed Definitions

Technical Background:

Blockchain

A Blockchain can be seen as a distributed database, shared across many computers in a network, as the name suggests, this database is built by subsequent blocks, each chained to the next one.

Block - refers to the data and state being stored. Blocks hold batches of valid transactions that are hashed and encoded into a cryptographic based data structure that chain the block to the previous block. When a transaction is executed, its data needs to be added to a block in order to count the transaction as successful. In addition to the transactions data, each block contains its characteristics like timestamp, root hash etc.

Every computer in the network must agree upon each new block and the chain as a whole. These computers are known as "nodes". Nodes ensure everyone interacting with the blockchain has the same data. To accomplish this distributed agreement, blockchains need a consensus mechanism[1].

Transaction

Transactions are cryptographically signed instructions between accounts. An account will initiate a transaction to update the state of the network[2]. Since executing and validating transactions and

[1] https://ethereum.org/developers/docs/intro-to-ethereum/
[2] https://ethereum.org/developers/docs/transactions/

blocks demands compute resources, typically in blockchain based ecosystems a fee is attached to each transaction, acting as a reward for the worker.

Ethereum

Ethereum is an open, public blockchain launched in 2015 by a software developer called Vitalik Buterin and a small team of co-founders. The idea behind Ethereum was simple. While Bitcoin lets you send and receive digital cash, Ethereum would build on this with open-source programs called **smart contracts.**

In the Ethereum universe, there is a single, canonical computer (EVM) whose state all nodes agree on, and keeps a copy of.

Any participant can broadcast a request for this computer to perform arbitrary computation. Whenever such a request is broadcast, other participants on the network verify, validate, and carry out ("execute") the computation. This execution causes a state change in the EVM, which is committed and propagated throughout the entire network. This request is essentially a transaction request.

Smart Contract[3]

A transaction protocol intended to automatically execute, control or document events and actions according to the terms of a contract or an agreement, this concept was originally created in 1996 and became the fundamental building block of the Ethereum network.

In the Ethereum ecosystem. Smart contracts are a reusable program written in Solidity, that runs on the EVM. Users can deploy new contracts or requests to execute an existing one with varying parameters.

Each contract contains data (state) + behaviour (functions)

Nodes and Clients[4]

A node is an instance of Ethereum client software that is connected to other computers also running Ethereum software, forming a network.

A client is an implementation of Ethereum that verifies data against the protocol rules and keeps the network secure. A node has to run two clients: a consensus client and an execution client:

The execution client listens to new transactions broadcasted in the network, executes them in EVM, and holds the latest state and database of all current Ethereum data.

The consensus client implements the consensus algorithm, which enables the network to achieve agreement based on validated data from the execution client.

In our context, Geth is an implementation of an execution client, we will focus on this part of Ethereum's execution layer, our implementation must follow the execution specs.

[3] https://ethereum.org/developers/docs/smart-contracts/
[4] https://ethereum.org/developers/docs/nodes-and-clients/

EVM - Ethereum Virtual Machine[5]

Up until now we described blockchain as a distributed database (or ledger) which lets its network's users to send and receive a decentralized currency with validations, Ethereum on top of that, implement the idea of smart contracts hence the Ethereum network can be think of as a distributed state machine, the state changes from block to block based on a pre-defined set of rules, those rules are defined by the EVM.

The EVM can be thought as a state transition function $f(S,T) = S'$ where:

S is the old valid state - multiple modified Merkle Patricia Trie which keeps all accounts and transactions linked by hashes and reducible to a single root hash, stored on the blockchain.

T is a set of transactions - in the context of Ethereum, there are 2 types of transactions, **contract creation transactions**, which are deployment of new smart contracts containing the compiled code of the contract. The other type is **message transactions**, which are requests to execute the contract's code when sent to the contract account.

S' is the new state produced by the transition function.

The EVM executes as a stack machine, where each compiled smart contract is executed as a number of EVM opcodes, which perform standard stack operations like XOR, AND, ADD, SUB, etc. each operation costs gas (Ethereum's name for the network fee) and during the execution the EVM updates the transient storage, a key-value data structure that does not get committed to the global state of the network, at the end of execution the trie associated with the account in question and part of the global state updated, those tries included in the contract.

The specs for the EVM are documented in Ethereum's yellow paper, this paper specifies the exact description of how the EVM works, note that Ethereum also runs the EIP process, which can add improvements to the yellow paper.

1.2 Operating platforms

Geth is maintained with build support for all common OSs and archs - Linux(ARM/32/64), Windows (32/64bit) and MacOS(Apple Silicon/Intel).

This project will be mainly focused on the core component code, and by the nature of Golang, which abstracts out most of the differences between the platforms, we anticipate our work to be portable to all the platforms specified above.

1.3 Interfaces

We will use native Go while utilizing existing components, protocols and interfaces that already exist in the Geth codebase.

[5] https://ethereum.org/developers/docs/evm/

2 Goal

Under our assumptions (specified below), we aim to achieve a speedup in the block execution time without harming the correctness of the outputs, and without transactions running out of gas in comparison to a sequential execution.
Moreover, our modification to Geth should be aligned with Ethereum's execution specs.
The main goal is to provide a POC, under some assumptions, for a potential drastic increase in Ethereum's network throughput.

3 Dependencies

Go programming language.

Ethereum's protocol and newly accepted EIPs.

Any dependencies that are already part of the Geth project.

4 Assumptions

Detecting dependencies between transactions inside a block can be a challenging mission, when a transaction execution results in a smart contract execution, the contract's code itself may interact with other contracts, this behaviour cannot be detected efficiently, one must run the contract in order to discover it, which makes the detection problem a NP-hard problem.

Thus, we will make our first assumption to ease the difficulty of the detection problem - the following EIP was introduced in 2021 to the Ethereum protocol:

EIP-2930[6] - Adds a transaction type which contains an access list, a list of addresses and storage keys that the transaction plans to access. Accesses outside the list are possible, but become more expensive.

This access list (AL) is optional and not all transactions include it in real network usage nowadays, even though a transaction benefits from including it by lowering its execution price, sometimes it may be too expensive to compute the AL. **We will assume that including the AL is mandatory.**

We will additionally assume there is no usage of the shared coinbase resource which allows the contract to modify its own control flow based on its value. This assumption is critical as transactions can decide how much they will contribute to this value and so predicting its value in OOO execution is impossible without running the contracts sequentially first.

[6] https://eips.ethereum.org/EIPS/eip-2930

5 Related Skills Evaluation

5.1 Previous Experience

This project builds on a proof of concept developed as a final project in the previous semester, where we successfully parallelized the execution of simple contracts. In parallel, Itay is currently pursuing a master's thesis focused on an asynchronous approach to pipeline parallelism for training deep learning models, while Ido is in his final semester of software engineering, he took the Concurrent and Distributed Programming course, 2 seminars in the context of concurrent programming. Together, we bring hands-on experience in transforming serial programs into parallel implementations, along with a solid understanding of parallel execution paradigms, shared memory architectures, and synchronization mechanisms.

5.2 Learning "Theory"

1. General blockchain and smart contracts understanding.

2. In-depth understanding of Ethereum execution model and protocols.

3. Geth previous design decisions in order to understand what approaches are possible, which already failed, and which were kept out for the sack of simplicity, and can still be utilized.

5.3 Learning Frameworks

1. Go programming language

2. Ethereum testing approach + existing blockchain testing frameworks

May also need to learn

1. Smart contracts development frameworks (Solidity)